

Research Methodology in CS (CSEG3060_3)

Lecture 7: IPR in Computer Software - copyright vs patent protection for software, software licensing models, and open-source issues

Unit 6: Patent Rights and New Developments in IPR • Mapped Course Outcome: CO5

Lecture 7: IPR in Computer Software — Copyright vs Patent Protection, Software Licensing Models, and Open-Source Issues

Course: Research Methodology in Computer Science (CSEG3060_3) **Program:** B.Tech (Computer Science) **Semester:** V **Unit:** 6 — Patent Rights and New Developments in Intellectual Property Rights **Mapped Course Outcome:** CO5 — Demonstrate an understanding of intellectual property rights, research ethics, and the legal frameworks governing innovation in computing. **Lecture Duration:** 60 minutes (Theory with Case Analysis)

1. Lecture Overview

Computer software is one of the most economically valuable and legally contested categories of intellectual property (IP) in the modern knowledge economy. Unlike physical inventions, software straddles the conceptual boundary between literary works (which fall naturally under copyright) and technological inventions (which traditionally fall under patent law). This dual character has produced a complex and evolving body of law, doctrine, and licensing practice.

This lecture develops a structured understanding of how intellectual property law protects software, beginning with the two principal regimes — **copyright** and **patent** — and then examining the practical mechanisms through which rights are transferred and shared: **software licensing models**. The second half of the lecture turns to the **open-source movement**, which has reshaped the economics and ethics of software creation, and to the unresolved legal tensions surrounding copyleft licensing, license compatibility, and the commercial exploitation of open code.

By the end of this lecture, students will be able to distinguish the protection scope, procedural requirements, and economic implications of copyright and patent law as applied to software, evaluate the trade-offs among licensing models, and critically engage with the open-source paradigm in a research and industry context.

2. Learning Objectives

Upon successful completion of this lecture, the learner will be able to:

1. **LO-7.1:** Explain the legal basis for copyright and patent protection of computer software under Indian and international law.
 2. **LO-7.2:** Differentiate the subject matter, scope, and limitations of copyright versus patent protection for software.
 3. **LO-7.3:** Classify and critically evaluate the major software licensing models used in industry and academia.
 4. **LO-7.4:** Analyze the philosophical, legal, and commercial implications of open-source licensing, including copyleft and permissive models.
 5. **LO-7.5:** Apply the legal framework to practical scenarios involving software IP disputes, licensing decisions, and open-source compliance.
-

3. Key Concepts and Terminology

The following terminology must be internalised before proceeding:

- **Computer Program:** A set of instructions expressed in any form, capable of causing an electronic device to perform a specific function (per Indian Copyright Act, 1957, Section 2(ffc)).
- **Source Code:** The human-readable form of a program written in a programming language.
- **Object Code:** The machine-executable, compiled form of a program, generally not human-readable.
- **Algorithm:** A finite sequence of well-defined instructions for solving a class of problems or performing a computation.
- **Look-and-Feel:** The overall visual and interactive appearance of a software interface, sometimes the subject of copyright or design litigation.
- **API (Application Programming Interface):** A defined set of protocols for inter-software communication; its copyrightability has been the subject of landmark litigation.
- **Licence:** A grant of permission by the rights-holder to use the protected work under defined conditions, without transferring ownership.
- **Open Source:** Software distributed under a licence that permits use, modification, and redistribution, subject to conditions defined by the licence.
- **Copyleft:** A licensing mechanism that requires derivative works to be distributed under the same licence terms, ensuring perpetual openness.

- **Permissive Licence:** A licence that allows derivative works to be distributed under different terms, including proprietary licences.
-

4. Introduction: Why Software IP is Unique

Software occupies a peculiar position in intellectual property law for three structural reasons:

1. **It is textually a literary work.** Source code is written in formal languages that resemble, in expressive character, technical writing or music. This makes it a natural subject for copyright.
2. **It is functionally a technological invention.** A program causes a machine to perform novel operations, solves technical problems, and may produce patentable technical effects. This brings it within the domain of patent law.
3. **It is dual-layered.** Both expressive (source code, UI design) and functional (algorithms, architectures, system behaviour) elements coexist within a single artefact. No single IP regime is fully adequate.

The policy challenge is to balance:

- **Incentives** for innovation and investment in software R&D.
- **Access** to foundational computing knowledge for cumulative research.
- **Disclosure** of inventions versus **protection** of trade secrets.
- **Standardisation** in interoperability versus **exclusion** in proprietary control.

The WIPO and TRIPS frameworks have left much of this balance to national legislatures and courts, producing significant divergence across jurisdictions.

5. Copyright Protection for Software

5.1 Legal Basis

In India, computer programs are protected as **literary works** under Section 13(1)(a) of the **Copyright Act, 1957**. The definition in Section 2(ffc) expressly includes "a set of instructions expressed in any form, capable of causing an electronic device to perform a specific function." Internationally, software has been protected under the **Berne Convention for the Protection of Literary and Artistic Works (1886)**, with the explicit recognition of computer programs as literary works under the **WIPO Copyright Treaty (WCT), 1996**.

5.2 Subject Matter of Protection

Copyright protects the **expression** of an idea, not the idea itself (the idea-expression dichotomy established in *Baker v. Selden*, 101 U.S. 99 (1879)). For software, this means:

- **Protected:** Source code, object code, the specific arrangement and selection of code, screen displays (in some jurisdictions), and preparatory design material.
- **Not Protected:** The underlying algorithm, the idea, the function performed, the system architecture as an abstract concept, and methods of operation.

5.3 Scope and Limitations

The owner of copyright in software has the exclusive right to:

- Reproduce the program in any material form.
- Issue copies to the public.
- Perform the program in public.
- Communicate the program to the public.
- Translate, adapt, or arrange the program (which includes porting, converting, or extracting parts).

Fair use (Section 52 of the Indian Copyright Act, 1957) permits limited use for purposes such as:

- Private and personal use, including making backup copies.
- Non-commercial research and criticism or review.
- Testing, debugging, and ensuring interoperability (reverse engineering under specified conditions).
- Use by educational institutions and libraries.

5.4 Term of Protection

In India, copyright in software subsists for **the lifetime of the author plus 60 years** from the year of the author's death (Section 22 of the Copyright Act, 1957). For anonymous, pseudonymous, or government works, the term is 60 years from the date of publication.

5.5 Advantages of Copyright Protection

- **Automatic acquisition:** No registration required; protection arises upon creation and fixation.
- **Low cost:** Substantially cheaper than patent prosecution.

- **Long effective term:** Coverage typically exceeds the commercial life of most software products.
- **Broad subject matter:** Almost any software qualifies.

5.6 Limitations of Copyright Protection

- Does **not** protect ideas, algorithms, or functional concepts — competitors may independently develop functionally equivalent code.
 - Cannot prevent independent creation, only copying.
 - Difficulty in proving copying of non-literal elements (structure, sequence, organisation).
-

6. Patent Protection for Software

6.1 Legal Basis

The grant of patents for software has been one of the most contested developments in modern IP law. Under the **Indian Patents Act, 1970**, "computer programs per se" were historically excluded from patentability under Section 3(k), which disallowed patents on "a computer program per se or algorithms." However, the **Patents (Amendment) Act, 2002** and the judicial interpretation in *State of Street*—style reasoning, as adopted by Indian courts, allow patenting when software produces a "technical effect" or constitutes a "technical solution to a technical problem."

In the United States, the Supreme Court's decision in *Alice Corp. v. CLS Bank International*, 573 U.S. 208 (2014), established the two-step test (the **Alice/Mayo framework**) for assessing software patent eligibility: (1) whether the claim is directed to a patent-ineligible abstract idea, and if so, (2) whether there is an "inventive concept" transforming it into a patent-eligible application. This test has been applied in numerous software patent cases globally.

6.2 Subject Matter of Patent Protection

Patent protection, when granted, covers:

- The novel, non-obvious, and useful **method or process** implemented by software.
- The **technical effect** produced (e.g., improved database query performance, image processing method, encryption technique).
- The **system architecture** when tied to a specific technical implementation.

Not protected:

- Pure algorithms or abstract mathematical methods.

- Business methods implemented solely on a generic computer.
- Computer programs per se without a claimed technical contribution.

6.3 Requirements for Patentability

A software-related invention must satisfy:

1. **Novelty** — not anticipated by prior art.
2. **Inventive Step (Non-Obviousness)** — not obvious to a person skilled in the art.
3. **Industrial Application (Utility)** — capable of industrial use.
4. **Disclosure** — sufficient to enable a person skilled in the art to perform the invention.
5. **Exclusion avoidance** — must not fall within Section 3(k) or other exclusions under Indian law.

6.4 Advantages of Patent Protection

- Protects the **functional and conceptual** aspects of software that copyright cannot.
- Allows infringement claims against independent re-implementations.
- Provides a strong barrier to entry and potential licensing revenue.

6.6 Limitations and Risks

- **Cost and time:** Patent prosecution is expensive and may take several years.
 - **Uncertain scope:** Particularly under Section 3(k) interpretation in India and Alice in the US.
 - **Disclosure obligation:** The patent specification requires full disclosure, which may benefit competitors.
 - **Validity challenges:** Software patents are frequently litigated and sometimes invalidated.
-

7. Copyright vs Patent Protection for Software: A Comparative Analysis

Dimension	Copyright	Patent
Legal basis in India	Copyright Act, 1957 (Sec 13(1)(a), Sec 2(ffc))	Patents Act, 1970 (Sec 3(k) carve-outs apply)
Subject matter	Expression (code, UI, structure)	Functional invention (process, technical effect)
What is protected	Specific written form of code	Idea, method, and underlying technical contribution
Acquisition	Automatic upon creation	Requires application and grant
Term	Life of author + 60 years	20 years from filing date
Cost	Low (registration optional)	High (filing, prosecution, maintenance)
Disclosure required	No	Full disclosure in specification
Independent creation defence	Yes — innocent independent creation is not infringement	Limited — if it falls within claims, independent creation still infringes
International harmonisation	Berne Convention; WCT	TRIPS Article 27 (with national exclusions)
Suitability for software	Strong, universally accepted	Conditional; subject to technical-effect test

7.1 When to Use Which

In practice, software developers rely primarily on copyright for everyday protection. Patent protection is pursued strategically for:

- Foundational algorithms with broad commercial value.
- Methods that solve novel technical problems.
- Software-implemented business methods that meet the inventive-step threshold (where permitted).
- Building a defensive or offensive patent portfolio.

8. Software Licensing Models

A software licence is a legal instrument governing the use, modification, redistribution, and reverse engineering of software. The licence determines the contractual relationship between the rights-holder (licensor) and the user (licensee). Several models coexist in industry practice.

8.1 Proprietary (Closed-Source) Licensing

The licensor retains all rights, the licensee is granted limited permission to use the software under defined restrictions, and the source code is not disclosed. Examples include Microsoft Windows (EULA), Adobe Photoshop, and most commercial enterprise software. Terms typically prohibit modification, redistribution, and reverse engineering.

8.2 Freeware

The software is distributed at no cost, but source code is not disclosed, and modification or redistribution is generally prohibited. The licensor retains copyright and full commercial rights.

8.3 Shareware

The software is distributed on a try-before-you-buy basis. The user is expected to pay a licence fee after a trial period. WinZip historically used this model.

8.4 Commercial / Enterprise Licensing

Software is sold under volume licences, per-seat licences, per-CPU licences, or subscription agreements (perpetual or term-based). Enterprise licences typically include service-level agreements (SLAs) and indemnification clauses.

8.5 Software as a Service (SaaS) Licensing

Under SaaS, software is accessed over the internet without local installation. The licence grants access rather than ownership. The economics shift from licence fees to recurring subscription revenue. Major examples include Salesforce, Microsoft 365, and Google Workspace.

8.6 Dual Licensing

The same software is offered under two licences: one open-source and one commercial. Companies may use the open-source version freely under the open-source licence, or purchase a commercial licence if they wish to incorporate the software into proprietary products. This model is used by MySQL (under GPL and a commercial licence by Oracle), Qt, and others.

8.7 Creative Commons (Less Common for Software)

Although primarily designed for creative works, some Creative Commons licences (e.g., CC BY, CC BY-SA) have been adapted for documentation and ancillary content associated with software, though they are not recommended by the Free Software Foundation for software code itself.

8.8 Licensing and Transfer of Rights

It is important to distinguish between:

- **Assignment (Transfer of Ownership):** A complete transfer of IP rights from the owner to another party, after which the assignee becomes the new owner. Requires written and, in some jurisdictions, registered instruments.
- **Licensing (Grant of Permission):** The owner retains ownership but grants defined rights to use the IP. Licences may be exclusive or non-exclusive, and may be royalty-bearing or royalty-free.

The decision between assignment and licensing has significant implications for the original creator, particularly in employment contexts and in academic research collaborations, where licensing is typically preferred to preserve residual rights.

9. Open-Source Software: Philosophy, Licences, and Issues

9.1 Free Software vs Open Source

Two distinct but largely overlapping movements underpin open-source software:

- **Free Software Movement** (Richard Stallman, Free Software Foundation, 1983): Defines software freedom in ethical terms. The four essential freedoms (as defined in the GNU Operating System documentation) are:
 - **Freedom 0:** Run the program for any purpose.
 - **Freedom 1:** Study how the program works and adapt it to your needs.
 - **Freedom 2:** Redistribute copies so you can help others.
 - **Freedom 3:** Improve the program and release improvements to the public.
- **Open Source Initiative (OSI), 1998:** Defines open source by ten operational criteria in the Open Source Definition, focusing on collaborative development and business-friendly licensing rather than ethical freedom.

The distinction is philosophical, not merely technical. Free software is a matter of user liberty; open source is a matter of development methodology.

9.2 Major Open-Source Licences

A. GNU General Public Licence (GPL) — Strong copyleft.

- Version 3 (2007) and version 2 (1991) are the dominant variants.
- Any derivative work distributed to others must also be licensed under the GPL.
- Imposes source disclosure for distributed copies.
- Used by the Linux kernel, GCC, WordPress (GPLv2+), and many others.

B. GNU Lesser General Public Licence (LGPL) — Weak copyleft.

- Allows linking from proprietary code without forcing the proprietary code itself to be open.
- Used by many libraries (e.g., GNU C Library).

C. GNU Affero General Public Licence (AGPL) — Network copyleft.

- Closes the SaaS loophole: even software accessed over a network (not distributed) triggers source disclosure obligations.

D. MIT Licence — Permissive.

- Permits use, modification, and redistribution in proprietary software, provided the copyright notice is preserved.
- Very short and business-friendly.

E. BSD Licence (2-clause and 3-clause variants) — Permissive.

- Similar to MIT; widely used in academic and embedded systems software.

F. Apache Licence 2.0 — Permissive with patent grant.

- Includes an explicit patent licence from contributors.
- Requires preservation of notices and a NOTICE file.
- Used by the Apache Foundation projects (Hadoop, Kafka, Spark).

G. Mozilla Public Licence (MPL) 2.0 — File-level copyleft.

- Modifications to MPL-licensed files must be released under MPL, but combining them with proprietary code is permitted.

9.3 Copyleft vs Permissive: Practical Implications

- **Copyleft licences** (GPL, AGPL) preserve openness downstream — a mechanism for ensuring that improvements remain available to the community. Critics argue this discourages commercial adoption.
- **Permissive licences** (MIT, BSD, Apache) encourage adoption but allow proprietary forking. Critics argue this enables free-riding on community labour.
- **Hybrid models** (LGPL, MPL) attempt to balance these concerns at the file or library level.

9.4 Open-Source Issues in Practice

A. Licence Compatibility Not all open-source licences are compatible. For example, GPLv2 and GPLv3 are incompatible in some combination scenarios. Code licensed under GPL cannot be relicensed under Apache 2.0 alone. Licence incompatibility is a major source of legal and engineering complications in large projects.

B. Patent Retaliation and Patent Peace The Apache Licence 2.0 and GPLv3 include patent retaliation clauses that terminate the licence if the licensee initiates patent litigation against contributors. This is intended to ensure patent peace in open communities.

C. Trademark Issues Open-source licences do not automatically grant trademark rights. Projects such as Linux, Firefox, and OpenOffice rely on separate trademark policies to prevent misuse of names, logos, and identities. The "Open Source Trademark" cases (e.g., the long-running *Linux Mark* disputes) demonstrate that trademark and copyright operate as separate regimes.

D. SaaS and the AGPL Question The SaaS delivery model has reopened the question of what constitutes "distribution" in the internet age. The AGPL was designed precisely to ensure that code served over the network remains open. Some major platforms (e.g., MongoDB's SSPL) have introduced bespoke licences restricting SaaS use.

E. Contributor Agreements and Joint Authorship For a copyright licence to be effective, contributors must hold or be authorised to license the copyright they contribute. Many foundations (Apache Software Foundation, Linux Foundation) require a **Contributor Licence Agreement (CLA)** to clarify the licensing status of contributions.

F. License Audits and Compliance Enterprises using open-source software must conduct licence audits to ensure compliance. Failure to comply — for instance, by using GPL code in a closed-source proprietary product — can result in copyright infringement claims and the loss of licence rights.

9.5 Business Models in Open Source

Open-source software has generated viable business models, including:

- **Support and services** (Red Hat, Canonical).
 - **Open-core:** Proprietary extensions to open-source bases (GitLab, MongoDB originally).
 - **SaaS hosting** (GitHub, WordPress.com).
 - **Dual licensing** (MySQL, Qt).
 - **Certification and training.**
 - **Foundational ecosystem sponsorship** (corporate-backed foundations).
-

10. Case Studies

Case Study 1: *Oracle America, Inc. v. Google, Inc.* (U.S., 2010–2021)

Oracle sued Google for copyright infringement after Google used Oracle's Java APIs in Android without permission. The central legal question was whether **APIs are copyrightable**. The District Court initially ruled APIs were not copyrightable; the Federal Circuit reversed; the Supreme Court eventually remanded. In its 2021 decision, the Supreme Court held that Google's use of the Java API declarations constituted fair use.

Lesson: The copyrightability of software interfaces remains unsettled in some jurisdictions. In India, the **Copyright (Amendment) Act, 2012** and the IT Act, 2000 do not yet contain clear API-specific provisions, leaving room for judicial development.

Case Study 2: *Microsoft Corporation v. Yogesh Papat* (India, 2005)

An Indian court addressed the unauthorised use and distribution of Microsoft software. The case established that pirated copies constitute infringement under the Indian Copyright Act and may attract both civil and criminal penalties under Section 63 of the Copyright Act, 1957.

Lesson: Software piracy is treated seriously under Indian law and may result in imprisonment and significant fines.

Case Study 3: *Linux Mark Institute v. Dr. Hans Werner and Related Linux Trademark Disputes*

The holder of a US trademark registration for "Linux" attempted to enforce it against distributors. The community, organised through the Linux Foundation, opposed the registration. Eventually the registration was cancelled or transferred to the foundation, securing community stewardship of the mark.

Lesson: Trademark rights in open-source contexts require active governance separate from copyright licensing.

Case Study 4: The Cambridge Centre for Alternative Finance and Free/Libre Open Source Software Adoption in India

Studies have documented widespread adoption of open-source stacks in Indian enterprise IT and academic research. Compliance with licences such as GPL remains a challenge, particularly when companies repackage and distribute GPL code without source disclosure.

Lesson: Open-source adoption without licence compliance creates latent legal exposure.

11. Practical Examples

Example 1: Choosing Protection for a Novel Image-Compression Algorithm

A team develops a novel image-compression algorithm and releases it as a software library. They should consider:

- **Copyright** for the code itself.
- **Patent** for the underlying compression method if it satisfies novelty, inventive step, and industrial application.
- **Trade secret** if the algorithm is not disclosed but used internally. A strategic portfolio combines copyright registration (in jurisdictions where this confers additional remedies) with patent filing in markets of commercial interest.

Example 2: Using GPL Code in a Proprietary Product

A startup integrates a GPL-licensed library into a closed-source SaaS product distributed to customers. Legal analysis:

- If distributed to users (i.e., users receive a copy), GPL obligations trigger source disclosure.
- If served only over a network (pure SaaS), the GPL does not trigger, but the AGPL would.
- Many companies choose permissive licences (MIT, Apache) to avoid these constraints.

Example 3: Academic Research Collaboration and IP Ownership

Two university research groups collaborate on an open-source ML framework. Issues to address in collaboration agreements:

- Who holds copyright in contributions?

- Is a CLA in place?
 - Which licence governs the combined work?
 - How are inventions (patentable subject matter) handled?
 - How are student contributors addressed under university IP policies?
-

12. Summary

- Computer software is protected primarily through **copyright** (as a literary work) and, where applicable, **patent** (as a technical invention).
 - **Copyright** protects the expression of software, arises automatically, and lasts the lifetime of the author plus 60 years under Indian law.
 - **Patent** protection for software is conditional upon demonstrating a technical effect and avoiding the Section 3(k) exclusion in India and the Alice/Mayo framework in the US.
 - **Software licensing models** govern the contractual use, modification, and redistribution of software, ranging from proprietary EULAs to open-source frameworks.
 - **Open-source licences** fall broadly into **copyleft** (GPL, AGPL, MPL) and **permissive** (MIT, BSD, Apache) families, with significant practical and philosophical distinctions.
 - **Open-source issues** include licence compatibility, patent retaliation, trademark governance, contributor agreements, and SaaS-related tensions.
 - Practical IP strategy for software combines the appropriate IP regime(s), a carefully chosen licensing model, and active compliance management.
-

13. Examination Preparation Tips

13.1 Common Question Patterns

1. **Distinguish** between copyright and patent protection for software (10–15 marks).
2. **Explain** the various software licensing models with examples (10 marks).
3. **Discuss** the open-source movement and the legal implications of the GPL (15 marks).
4. **Examine** the scope of Section 3(k) of the Indian Patents Act in relation to software (10 marks).
5. **Analyse** the issues surrounding open-source licensing in SaaS environments (15 marks).

13.2 Higher-Order Practice Questions

1. A startup has developed an AI-based fraud detection algorithm. Outline an IP strategy that combines copyright, patent, trade secret, and licensing considerations.

2. Two open-source communities wish to merge their codebases, one under Apache 2.0 and one under GPLv3. Identify the legal issues that may arise.
3. A research paper you have authored includes pseudocode for an algorithm that you intend to commercialise. Discuss the implications for academic publication and subsequent patentability.
4. Compare the approaches to software patentability in India (Section 3(k)) and the United States (Alice/Mayo). What practical advice would you give to an Indian software startup seeking patent protection abroad?

13.3 Revision Checklist

- Definition of computer program under the Indian Copyright Act.
- Copyright term: lifetime + 60 years.
- Section 3(k) and the technical-effect doctrine.
- Berne Convention and WIPO Copyright Treaty.
- Alice/Mayo two-step test for software patents.
- Four freedoms of free software.
- OSI Open Source Definition criteria (memorise at least five).
- Major licence families: GPL, LGPL, AGPL, MIT, BSD, Apache, MPL.
- Difference between copyleft and permissive licences.
- Difference between assignment and licence.
- Key cases: *Oracle v. Google*, *Baker v. Selden*, *Alice Corp. v. CLS Bank*.

13.4 Examination Strategy

- Use diagrams or tables to compare copyright and patent protection.
- Cite specific statutory provisions (e.g., Section 3(k), Section 13(1)(a), Section 52).
- Refer to landmark cases with citation format and year.
- Apply the law to a hypothetical scenario rather than reproducing text verbatim to score higher.
- Where appropriate, integrate ethical and policy arguments (incentive vs access, openness vs commercial returns).

14. References and Suggested Reading

1. Cornish, W. R., & Llewelyn, D. *Intellectual Property: Patents, Copyright, Trade Marks and Allied Rights*. Sweet & Maxwell.

2. Narayanan, P. *Intellectual Property Law*. Eastern Law House.
3. Bainbridge, D. *Intellectual Property*. Pearson.
4. Free Software Foundation. *GNU General Public Licence, version 3*.
<https://www.gnu.org/licenses/gpl-3.0.html>
5. Open Source Initiative. *The Open Source Definition*. <https://opensource.org/osd>
6. World Intellectual Property Organization. *WIPO Copyright Treaty*. Geneva, 1996.
7. Government of India. *The Copyright Act, 1957* (as amended).
8. Government of India. *The Patents Act, 1970* (as amended).
9. *Oracle America, Inc. v. Google, Inc.*, 593 U.S. ____ (2021).
10. *Alice Corp. v. CLS Bank International*, 573 U.S. 208 (2014).
11. *Baker v. Selden*, 101 U.S. 99 (1879).
12. Rosen, L. *Open Source Licensing: Software Freedom and Intellectual Property Law*. Prentice Hall.
13. Subbanna, S. & Reddy, P. *Indian Patent Law and Practice*. LexisNexis.

15. Mapping to Course Outcome

Lecture Component	Course Outcome (CO5)
Copyright vs Patent for software	Demonstrates understanding of IP regimes
Licensing models	Applies IP law to technology transfer scenarios
Open-source analysis	Engages with contemporary IP debates
Case studies	Develops analytical and applied reasoning
Practice questions	Prepares for research methodology evaluation of IP issues

This concludes Lecture 7. Students are expected to revise the comparative tables, review the statutory provisions cited, and attempt at least three practice questions before the next session.